

Decision Trees & Random Forests

From Interpretable Rules to Powerful Ensembles

■ TREE-BASED MODELS

■ LEARNING OUTCOME

By the end of this module you will understand how decision trees split data using Gini/entropy, build Random Forest and XGBoost ensembles, extract feature importances, and tune hyperparameters effectively.

■ Skills You Will Gain

- Explain how a decision tree chooses splits using Gini impurity
- Visualise and interpret a decision tree's decision rules
- Build a Random Forest (bagging ensemble)
- Implement XGBoost gradient boosting
- Extract and visualise feature importances
- Tune `n_estimators`, `max_depth`, `learning_rate`

■ Random Forests and XGBoost win 80% of Kaggle tabular competitions. Master these and you can solve almost any structured data problem with high accuracy.

SECTION 1

How Decision Trees Work

1	Start at Root All training samples start at the root node of the tree
2	Choose Best Split For every feature and threshold, compute information gain — pick the best
3	Gini Impurity $Gini = 1 - \sum(p_i^2)$. Value 0 = pure node (one class). Value 0.5 = maximum impurity
4	Information Gain $IG = \text{Parent_impurity} - \text{weighted_average}(\text{Child_impurity})$. Choose max IG split
5	Recurse Repeat split on each child node until stopping criterion is met
6	Leaf Nodes Final prediction: majority class (classification) or mean (regression)
7	Stopping Rules max_depth, min_samples_leaf, min_impurity_decrease prevent over-splitting

```
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text import matplotlib.pyplot as plt
tree = DecisionTreeClassifier( max_depth=4, # limit depth to prevent overfitting
min_samples_leaf=5, # each leaf must have >= 5 samples criterion='gini', # or 'entropy'
random_state=42 ) tree.fit(X_train, y_train) # Visualise plt.figure(figsize=(18, 8))
plot_tree(tree, feature_names=X.columns, class_names=class_names, filled=True, rounded=True,
fontsize=8) plt.savefig('tree.png', dpi=120, bbox_inches='tight') # Print text rules
print(export_text(tree, feature_names=list(X.columns)))
```

SECTION 2

Random Forest — Bagging Ensemble

Bagging = Bootstrap AGGREGatING

1. Draw B random samples WITH replacement from training data (bootstrap samples) 2. Train a full decision tree on each bootstrap sample — each sees a random subset of features at each split 3. Aggregate predictions: majority vote (classification) or average (regression) Why it works: Individual trees overfit in DIFFERENT directions. Averaging their errors cancels the noise.

```
from sklearn.ensemble import RandomForestClassifier import pandas as pd rf =
RandomForestClassifier( n_estimators=200, # number of trees – more is better (diminishing returns)
max_features='sqrt', # features per split (sqrt for classification, 1/3 for regression)
max_depth=None, # grow full trees – bagging prevents overfitting min_samples_leaf=2, n_jobs=-1, #
use all CPU cores in parallel random_state=42 ) rf.fit(X_train, y_train) # Feature importances
importances = pd.Series(rf.feature_importances_, index=X.columns)
importances.nlargest(15).sort_values().plot( kind='barh', color='#7C3AED', figsize=(10, 6),
title='Feature Importance')
```

SECTION 3

XGBoost — Gradient Boosting

vs Bagging	Boosting (XGBoost)
Trees built in PARALLEL	Trees built SEQUENTIALLY — each corrects the previous
Each tree sees random subset	Each tree focuses on samples with highest error
Reduces VARIANCE (overfitting)	Reduces BIAS (underfitting) — learns more complex patterns
Random Forest example	XGBoost, LightGBM, CatBoost examples
More robust to overfitting	Higher accuracy but needs careful tuning

```
from xgboost import XGBClassifier xgb = XGBClassifier( n_estimators=500, learning_rate=0.05, # shrinkage - lower = more trees, better generalisation max_depth=5, subsample=0.8, # row sampling per tree colsample_bytree=0.8, # feature sampling per tree reg_alpha=0.1, # L1 regularisation reg_lambda=1.0, # L2 regularisation eval_metric='logloss', random_state=42 ) xgb.fit(X_train, y_train, eval_set=[(X_val, y_val)], early_stopping_rounds=20, # stop if val score doesn't improve for 20 rounds verbose=50)
```

■ PRACTICE Q & A

Q1. What is Gini impurity? What value indicates a pure node?

A: $Gini = 1 - \sum(p_i^2)$ where p_i = proportion of class i . Value 0 = pure node (all one class). Value 0.5 = maximum impurity (equal class proportions).

Q2. Why does Random Forest generalize better than a single deep decision tree?

A: Multiple trees trained on random subsets vote together. Each tree overfits differently, so errors cancel out when aggregating predictions.

Q3. What is early stopping in XGBoost and why is it useful?

A: Stops training when validation metric stops improving for N rounds. Prevents overfitting without manually tuning $n_estimators$ — saves time and compute.

Q4. How do you get feature importances from a Random Forest and what do they mean?

A: `rf.feature_importances_` — average reduction in impurity (Gini/entropy) that each feature contributes across all splits in all trees.

Q5. When would you prefer XGBoost over Random Forest?

A: When you need maximum accuracy on tabular data and have time to tune. XGBoost typically wins by 1-3% accuracy but is more sensitive to hyperparameters.

■ Tree Models Cheat Sheet	
<code>DecisionTreeClassifier(max_depth=4)</code>	Single decision tree
<code>RandomForestClassifier(n_estimators=200)</code>	Bagging ensemble
<code>XGBClassifier(learning_rate=0.05)</code>	Gradient boosting
<code>LGBMClassifier()</code>	LightGBM — faster XGBoost
<code>CatBoostClassifier()</code>	Best for categorical features
<code>rf.feature_importances_</code>	Feature importance scores
<code>plot_tree(tree, filled=True)</code>	Visualise decision tree
<code>max_depth</code>	Controls tree complexity
<code>n_estimators</code>	Number of trees in ensemble
<code>early_stopping_rounds=20</code>	Stop when val score plateaus

<code>subsample=0.8</code>	Row sampling (anti-overfit)
<code>colsample_bytree=0.8</code>	Feature sampling per tree